

Checklist:

Team Topologies – Team-First Thinking

1. Team Structure

- ☐ Treat the **team**, not the individual, as the basic unit of delivery.
- ☐ Keep teams small, ideally **5–9 people**.
- ☐ Avoid assigning software work directly to individuals; assign work to teams.
- ☐ Keep teams **long-lived**, so they have time to form trust and effectiveness.
- ☐ Avoid frequent team reshuffling unless there is a clear need.
- ☐ Allow teams to evolve, but do not disrupt them casually.

2. Team Trust and Stability

- ☐ Design teams around high trust, psychological safety, and shared goals.
- ☐ Watch for signs that a team is too large: slower decisions, weaker trust, coordination friction.
- ☐ Split teams when size or complexity begins to damage effectiveness.
- ☐ Give new or changed teams time to pass through the forming, storming, norming, and performing stages.
- ☐ Support teams with coaching when they are newly formed or under strain.

3. Ownership of Software

- ☐ Ensure every software system, service, component, or subsystem is owned by **exactly one team**.
- ☐ Avoid shared ownership of code, libraries, or components.
- ☐ Allow other teams to contribute through pull requests, suggestions, or agreed collaboration paths.
- ☐ Make the owning team responsible for operability, maintenance, and evolution.

- ☐ Encourage teams to think of themselves as **stewards**, not private owners, of software.

4. Team-First Mindset

- ☐ Encourage team members to put the team's needs above their individual preferences.
- ☐ Make team goals visible and shared.
- ☐ Keep meetings, standups, and investigations focused.
- ☐ Encourage people to unblock teammates before starting new work.
- ☐ Mentor less experienced team members.
- ☐ Avoid “winning” arguments; explore options together.
- ☐ Address consistently toxic or anti-team behavior quickly.

5. Diversity and Inclusion

- ☐ Build teams with diverse backgrounds, skills, and perspectives.
- ☐ Encourage multiple viewpoints when solving problems.
- ☐ Watch for groupthink or overly homogeneous decision-making.
- ☐ Create space for quieter or newer team members to contribute.
- ☐ Treat disagreement as useful input, not disruption.

6. Rewards and Incentives

- ☐ Reward the **whole team**, not just individuals.
- ☐ Avoid incentives that create internal competition within the team.
- ☐ Share training budgets at the team level where possible.
- ☐ Evaluate outcomes based on team contribution and delivery.
- ☐ Make sure performance systems do not undermine collaboration.

7. Cognitive Load

- ☐ Regularly ask teams whether they feel effective and able to respond in time to the work they are assigned.
- ☐ Identify sources of **intrinsic cognitive load**: essential domain or technical complexity.
- ☐ Identify sources of **extraneous cognitive load**: unnecessary process, tooling, handoffs, confusion, or poor documentation.
- ☐ Protect space for **germane cognitive load**: learning, domain mastery, and problem-solving.

- ☐ Reduce unnecessary meetings, context switching, unclear ownership, and avoidable dependencies.
- ☐ Provide training, automation, better tooling, and documentation to reduce overload.

8. Team Responsibilities and Domains

- ☐ Identify the distinct domains each team is responsible for.
- ☐ Classify domains as simple, complicated, or complex.
- ☐ Assign each domain to a single team where possible.
- ☐ Do not assign a complex domain plus additional domains to one team.
- ☐ Avoid giving one team responsibility for multiple complicated domains.
- ☐ Split large domains into smaller subdomains before assigning them to teams.
- ☐ Review whether team responsibilities still match team capacity.

9. Software Boundaries

- ☐ Design software boundaries around the team's cognitive load, not just the technical architecture.
- ☐ Avoid choosing between monoliths and microservices before understanding the team's capacity.
- ☐ Keep software subsystems small enough for one team to understand and own.
- ☐ Align architecture with team ownership and communication patterns.
- ☐ Realign architecture when teams or business streams grow too large.
- ☐ Prefer clear, team-sized services or subsystems over tangled shared ownership.

10. Team APIs

- ☐ Define a clear "Team API" for every team.
- ☐ Include code, runtime endpoints, libraries, clients, or user interfaces owned by the team.
- ☐ Include versioning and compatibility expectations.
- ☐ Include documentation, how-to guides, and onboarding information.
- ☐ Include team working practices and communication channels.
- ☐ Include current priorities, roadmap, and upcoming work.
- ☐ Make it easy for other teams to interact without needing informal insider knowledge.
- ☐ Regularly test whether other teams find the Team API usable.

11. Team Interactions

- ☐ Provide time and space for teams to build trust with each other.
- ☐ Create opportunities for communities of practice, guilds, internal conferences, and learning sessions.
- ☐ Encourage cross-team awareness without creating constant interruption.
- ☐ Make collaboration paths explicit.
- ☐ Avoid depending only on personal relationships or informal networks.
- ☐ Support occasional cross-team discovery work without creating permanent dependency structures.

12. Physical and Virtual Environment

- ☐ Design workspaces for team collaboration, focused work, and cross-team interaction.
- ☐ Avoid both isolated cubicles and fully open-plan layouts as default solutions.
- ☐ Give teams identifiable areas or spaces.
- ☐ Provide whiteboards, collaboration areas, and quiet spaces.
- ☐ Make virtual spaces easy to navigate.
- ☐ Use predictable naming conventions for chat channels, wiki pages, and support spaces.
- ☐ Make it easy to identify who belongs to which team.
- ☐ Reduce digital noise and unnecessary notifications.

13. Engineering Practices

- ☐ Invest in continuous delivery.
- ☐ Use test-first or test-supported development practices.
- ☐ Treat operability and reliability as first-class concerns.
- ☐ Build delivery pipelines as important products in their own right.
- ☐ Ensure teams can safely release, observe, and operate their own software.
- ☐ Do not expect team-first design to work without strong engineering foundations.

14. Warning Signs to Watch For

- ☐ Teams are larger than 9 people, and trust is weakening.
- ☐ Multiple teams are changing the same codebase without clear ownership.
- ☐ Individuals, not teams, are treated as the main delivery unit.
- ☐ Teams are frequently reshuffled before they become effective.
- ☐ A team owns too many domains or too much software.
- ☐ Teams say they are overloaded, but leadership treats the issue as a staffing or motivation problem.

- ☐ Rewards encourage individual heroics over team outcomes.
- ☐ Other teams do not know how to interact with a team or its software.
- ☐ Architecture reflects old reporting lines instead of current team ownership.
- ☐ Collaboration depends on informal favors rather than clear interaction mechanisms.

15. Final Review Questions

- ☐ Is each team small, stable, and long-lived?
- ☐ Does each team own a clear part of the software?
- ☐ Is every part of the system owned by exactly one team?
- ☐ Are team responsibilities within cognitive load limits?
- ☐ Are software boundaries aligned to team boundaries?
- ☐ Do other teams know how to interact with this team?
- ☐ Are incentives aligned with team success?
- ☐ Is the physical or virtual environment helping teams collaborate?
- ☐ Are engineering practices strong enough to support fast, safe flow?
- ☐ Are teams able to deliver effectively without constant overload?

